Элементы криптографии

Однократное гаммирование

Лабораторная работа № 7 по основам информационной безопасности

Ицков Андрей Станиславович, НФИбд-03-24

12 октября 2026

Проверил: Кулябов Дмитрий Сергеевич, д.ф.-м.н., профессор кафедры теории вероятностей и кибербезопасности
Российский университет дружбы народов

Докладчик

Ицков Андрей Станиславович

Студент группы НФИбд-03-24

Российский университет дружбы народов

Основы информационной безопасности

Виртуальная машина и учётная запись: **asitskov**

Предмет исследования

- **Актуальность:** корректное представление данных необходимо для обратимого шифрования.
- **Объект:** текстовые сообщения и ключевые последовательности.
- **Предмет:** побайтовый XOR и однократная гамма.
- **Практическое значение:** проверка обратимости и требований к ключу.
- Работа исследует известный алгоритм. Научная новизна не заявляется [1].

Цель и задачи

Освоить однократное гаммирование.

- Реализовать C = P XOR K и обратное преобразование.
- Найти K для заданных C и выбранного P.
- Получить «С Новым Годом, друзья!» полностью.
- Проверить длины и операции по сохранённым файлам.

Гипотеза: для любого текста длиной с C можно вычислить согласованный ключ K = C XOR P.

Материалы и методы

- Rocky Linux 9.8, Python 3.9.25, VirtualBox.
- ВМ и пользователь asitskov.
- Текстовые данные в cp1251, исходник в UTF-8.
- argparse для команд, secrets для новой гаммы.
- HEX-файлы и побайтовая таблица.
- Проверки по записи выполнения 12 октября 2026 года.

Принцип гаммирования

Шифрование: C = P XOR K

Расшифрование: P = C XOR K

Вычисление ключа: K = C XOR P

Таблица 1: Таблица XOR

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

В табл. 1 одинаковые биты дают 0. Поэтому (P XOR K) XOR K = P [1].

- encrypt вычисляет шифротекст.
- decrypt восстанавливает текст.
- derive находит ключ для выбранного прочтения.
- genkey создаёт новую гамму нужной длины.
- Проверка длин выполняется **до zip**.
- –out сохраняет HEX и предотвращает перезапись.

Интерфейс реализован через argparse [2].

Компиляция завершилась с кодом **0**. Исходник содержит **110 строк**; контрольная сумма совпала.

Результат показан на рис. 1.

```
[asitskov@asitskov lab07-VMA0nF8m]$ python3 -m py_compile gamma.py
echo "compile exit=$?"
wc -l gamma.py
sha256sum gamma.py
python3 gamma.py --help
^[compile exit=0
110 gamma.py
21d3662214f0885c6f99391b67ccfa12e7c64ad88bf2de7fd14f673d4633a12c  gamma.py
usage: gamma.py [-h] {genkey,encrypt,decrypt,derive} ...

ЛР 7: XOR-гаммирование байтов

positional arguments:
  {genkey,encrypt,decrypt,derive}
    genkey              Создать случайную гамму

optional arguments:
  -h, --help            show this help message and exit
[asitskov@asitskov lab07-VMA0nF8m]$
```

Рисунок 1: Проверка программы и справки

Пример методички: **20 байт**. Полное поздравление: **22 байта cp1251**. Размеры UTF-8 отличаются [3].

Результат показан на рис. 2.

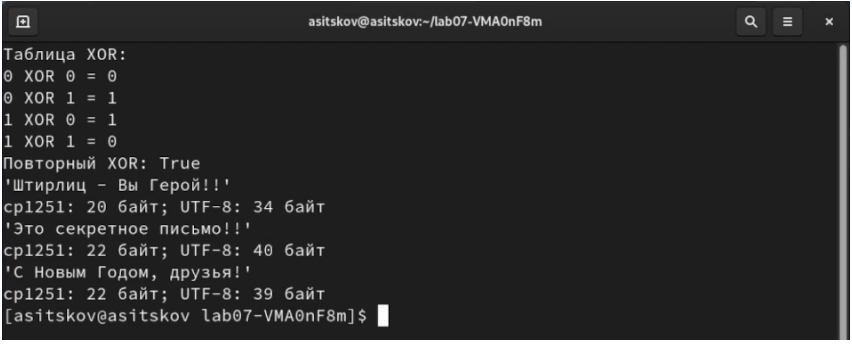


Рисунок 2: Длины строк и свойства XOR

Шифрование примера

По известным P и K вычислен C. Все 20 байт совпали с методичкой.

Результат показан на рис. 3.

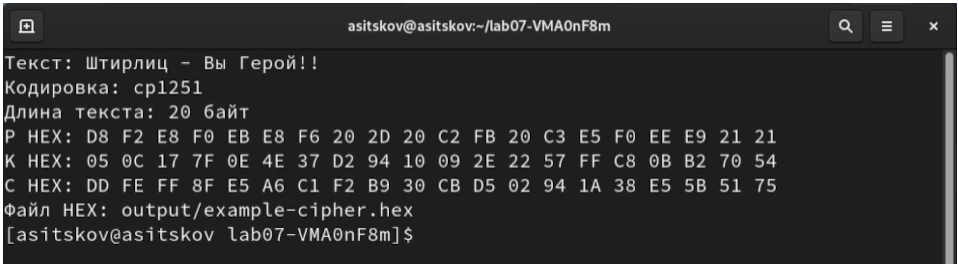


Рисунок 3: Текст и шифротекст примера

Повторный XOR восстановил **«Штирлиц - Вы Герой!!»**.

Результат показан на рис. 4.

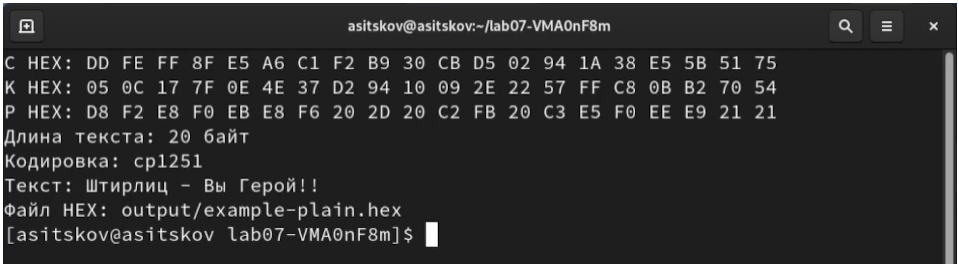


Рисунок 4: Обратное преобразование примера

При неизменном C вычислен ключ для **«Штирлиц - Вы Болван!»**. Проверка дала **True**.

Результат показан на рис. 5.

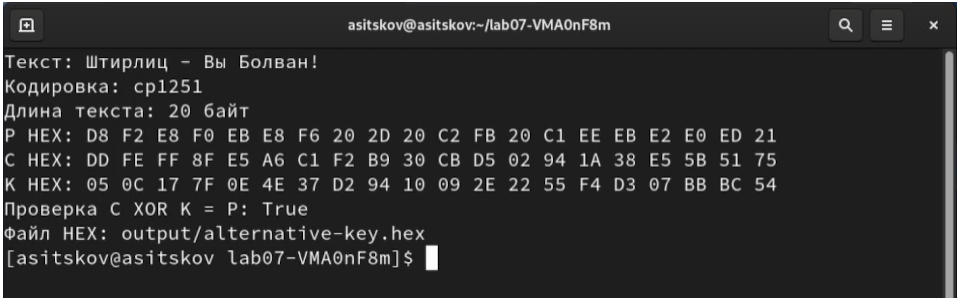


Рисунок 5: Вычисление ключа для другого текста

Другое прочтение

Тот же шифротекст с другим ключом дал другой полный текст. Возможное прочтение не устанавливает подлинный текст отправителя.

Результат показан на рис. 6.

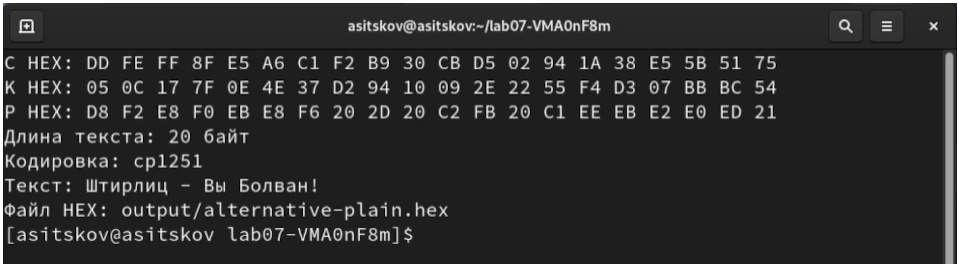


Рисунок 6: Подтверждение второго прочтения

Исходные данные для поздравления

В задании не указан отдельный 22-байтовый C. Создал его из **«Это секретное письмо!!»** и нового учебного ключа.

Результат показан на рис. 7.

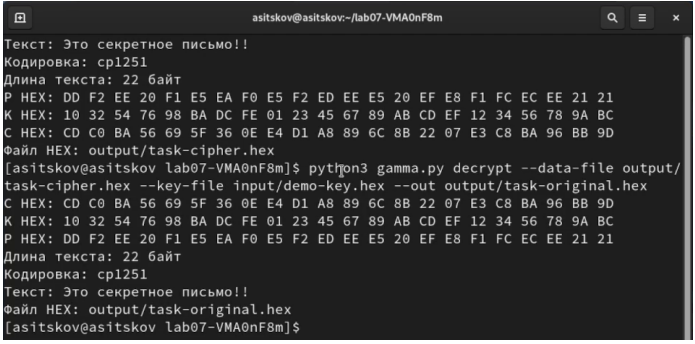


Рисунок 7: Новый шифротекст длиной 22 байта

Для C длиной 20 байт и P длиной 22 байта приложение вернуло **код 2**. Усечённый результат не создаётся.

Результат показан на рис. 8.

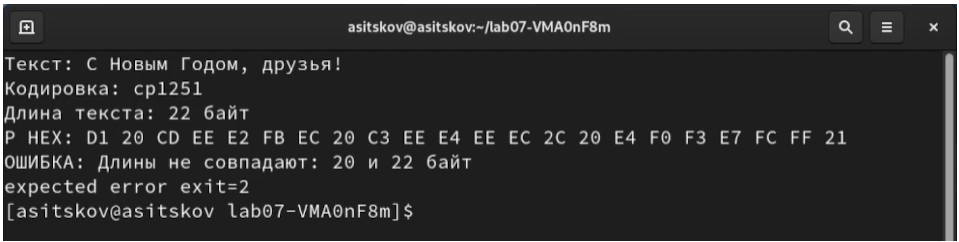


Рисунок 8: Контролируемый отказ 20 и 22 байта

K = C XOR P вычислен для всех **22 позиций**. Проверка C XOR K = P дала **True**.

Результат показан на рис. 9.

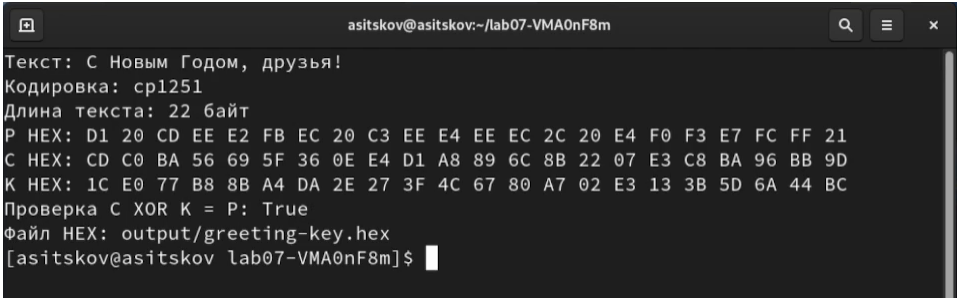


Рисунок 9: Полный ключ для заданной фразы

Требуемое сообщение

Найденный ключ превратил неизменный task-cipher.hex в **«С Новым Годом, друзья!»**.

Результат показан на рис. 10.

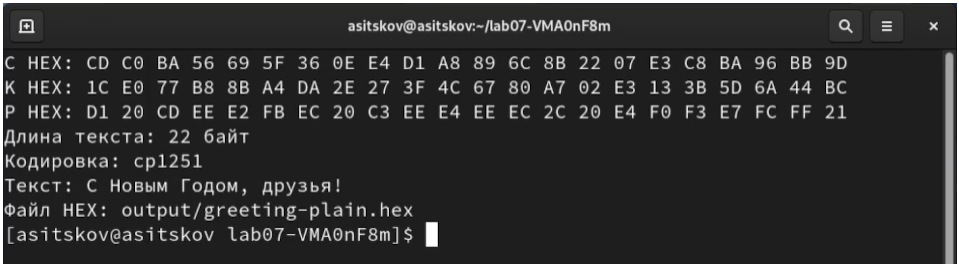


Рисунок 10: Главный результат работы

Таблица 2: Фрагмент таблицы вычисления ключа

Позиция	C HEX	P HEX	K HEX
1	CD	D1	1C
2	C0	20	E0
10	D1	EE	3F
21	BB	FF	44
22	9D	21	BC

В табл. 2 **K = C XOR P** для каждой позиции.

Полная таблица содержит 22 строки и приведена в отчёте.

Длины C, K, P: 22/22/22. Полное совпадение текста и оба обратных равенства дали True.

Результат показан на рис. 11.

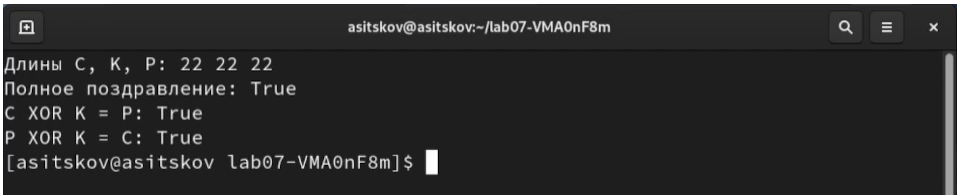


Рисунок 11: Проверка сохранённых файлов

secrets.token_bytes создал **22 новых байта**. Поздравление зашифровано отдельным ключом [4].

Результат показан на рис. 12.

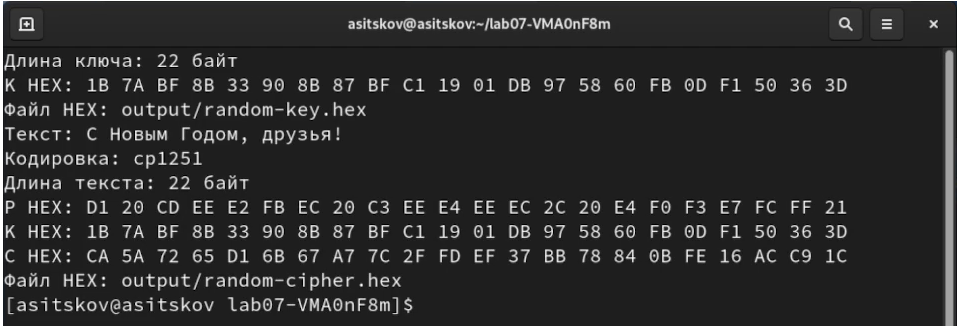


Рисунок 12: Случайная гамма фактического запуска

Восстановлено полное поздравление. Сравнение файлов завершилось с **cmp exit=0**.

Результат показан на рис. 13.

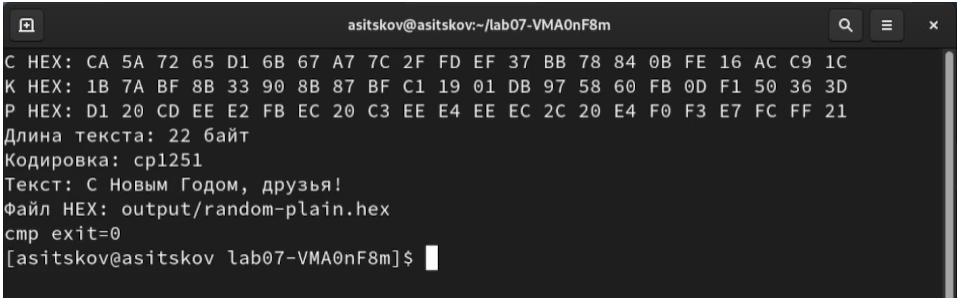


Рисунок 13: Совпадение результатов расшифрования

Условия стойкости

- Гамма секретная и независимая от сообщения.
- В идеальном блокноте она истинно и равномерно случайная.
- Длина применяемой гаммы равна длине сообщения.
- Для нового сообщения нужен новый ключ.
- XOR сам по себе не проверяет целостность.

Фиксированные ключи демонстрируют вычисления. Генератор ОС не доказывает теоретическую идеальную случайность [1,4].

Выводы

- Приложение выполняет три преобразования через один XOR.
- Пример методички полностью воспроизведён.
- Найден ключ для полного поздравления длиной 22 байта.
- Обратные проверки прошли, несовпадение длин отвергнуто.
- Случайная гамма также восстановила исходные данные.
- Цель работы достигнута.

Источники

1. Кулябов Д. С., Королькова А. В., Геворкян М. Н. Элементы криптографии. Однократное гаммирование. Российский университет дружбы народов.
2. argparse — Parser for command-line options [Электронный ресурс]. Python Software Foundation. URL: https://docs.python.org/3.9/library/argparse.html (дата обращения: 12.10.2026).
3. codecs — Codec registry and base classes [Электронный ресурс]. Python Software Foundation. URL: https://docs.python.org/3.9/library/codecs.html (дата обращения: 12.10.2026).
4. secrets — Generate secure random numbers [Электронный ресурс]. Python Software Foundation. URL: https://docs.python.org/3.9/library/secrets.html (дата обращения: 12.10.2026).